

TP 1: Analyse exploratoire des données

September 14, 2023

1 Introduction

L'analyse exploratoire des données (EDA) constitue la première étape cruciale dans la compréhension approfondie d'un jeu de données avant d'entreprendre des analyses plus complexes. Cette phase est essentielle pour révéler des informations cachées, identifier des tendances, détecter des valeurs aberrantes et permettre une meilleure prise de décision. Dans ce TP, nous explorerons en détail l'EDA en utilisant des outils puissants tels que Pandas et Seaborn.

Objectifs du TP :

- Familiarisation avec Pandas : Notre premier objectif sera de maîtriser l'utilisation de la bibliothèque Pandas, l'une des plus populaires pour la manipulation et l'analyse de données. Nous apprendrons à charger, examiner, trier et filtrer des données de manière efficace.
- Visualisation de Données : Nous étudierons diverses techniques de visualisation de données à l'aide de Seaborn, une bibliothèque de visualisation de données puissante et conviviale. Vous serez initié à la création de graphiques tels que les diagrammes en barres, les histogrammes, les diagrammes en boîte, et les graphiques de dispersion pour révéler des tendances et des motifs.
- Identification des Valeurs Aberrantes : Les valeurs aberrantes peuvent avoir un impact significatif sur l'analyse. Nous apprendrons à les détecter et à décider comment les gérer en toute connaissance de cause.
- Analyse des Relations : Nous examinerons comment explorer les relations entre différentes variables, notamment la corrélation entre les attributs, ce qui peut fournir des informations précieuses pour la modélisation ultérieure.
- Objectif de Modélisation : Enfin, nous comprendrons comment l'EDA peut nous aider à définir des objectifs de modélisation clairs et à identifier les variables pertinentes pour notre analyse.

À la fin de ce TP, vous disposerez des compétences essentielles en EDA nécessaires pour explorer, comprendre et tirer des informations significatives de n'importe quel ensemble de données. Préparez-vous à maîtriser les outils de Pandas et Seaborn, ainsi qu'à développer une intuition pour la manière dont les données peuvent être analysées et utilisées pour prendre des décisions éclairées.

2 Pandas

pandas est une bibliothèque logicielle écrite pour le langage de programmation Python pour la manipulation et l'analyse de données. En particulier, il propose des structures de données et des

opérations de manipulation de tableaux numériques et de séries chronologiques.

```
[1]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
plt.style.use('ggplot')
```

2.1 DataFrame Pandas

Données tabulaires bidimensionnelles, variables en taille et potentiellement hétérogènes. La structure de données contient également des axes étiquetés (lignes et colonnes). Les opérations arithmétiques s'alignent sur les étiquettes de ligne et de colonne. Peut être considéré comme un conteneur de type dict pour les objets Series. La structure de données principale des pandas

Charger vos données dans un Les options les plus courantes : - read_csv - read_excel

```
[2]: data = pd.read_excel('Datasets/titanic3.xls')
```

```
[3]: data.shape
data.head()
```

```
[3]:    pclass  survived                                name  sex \
0         1         1                Allen, Miss. Elisabeth Walton  female
1         1         1            Allison, Master. Hudson Trevor    male
2         1         0            Allison, Miss. Helen Loraine  female
3         1         0      Allison, Mr. Hudson Joshua Creighton    male
4         1         0  Allison, Mrs. Hudson J C (Bessie Waldo Daniels)  female
```

```
    age  sibsp  parch  ticket    fare    cabin embarked boat  body \
0 29.0000    0     0   24160  211.3375      B5      S     2   NaN
1  0.9167    1     2  113781  151.5500  C22 C26      S    11   NaN
2  2.0000    1     2  113781  151.5500  C22 C26      S   NaN   NaN
3 30.0000    1     2  113781  151.5500  C22 C26      S   NaN  135.0
4 25.0000    1     2  113781  151.5500  C22 C26      S   NaN   NaN
```

```
    home.dest
0      St Louis, MO
1  Montreal, PQ / Chesterville, ON
2  Montreal, PQ / Chesterville, ON
3  Montreal, PQ / Chesterville, ON
4  Montreal, PQ / Chesterville, ON
```

```
[4]: data.describe() # Avoir une analyse statistique des donnees
```

```
[4]:    count    pclass    survived    age    sibsp    parch \
count  1309.000000  1309.000000  1046.000000  1309.000000  1309.000000
mean     2.294882    0.381971   29.881135    0.498854    0.385027
std     0.837836    0.486055   14.413500    1.041658    0.865560
```

min	1.000000	0.000000	0.166700	0.000000	0.000000
25%	2.000000	0.000000	21.000000	0.000000	0.000000
50%	3.000000	0.000000	28.000000	0.000000	0.000000
75%	3.000000	1.000000	39.000000	1.000000	0.000000
max	3.000000	1.000000	80.000000	8.000000	9.000000

	fare	body
count	1308.000000	121.000000
mean	33.295479	160.809917
std	51.758668	97.696922
min	0.000000	1.000000
25%	7.895800	72.000000
50%	14.454200	155.000000
75%	31.275000	256.000000
max	512.329200	328.000000

2.2 Nettoyer des Datasets

La fonction “drop” supprime des lignes ou des colonnes en spécifiant les noms d’étiquette et l’axe correspondant, ou en spécifiant directement les noms d’index ou de colonne.

```
[5]: data = data.drop(['name', 'sibsp', 'parch', 'ticket', 'cabin', 'embarked',
↳ 'boat', 'body', 'home.dest'], axis=1)
```

Remplir les valeurs vides (NA/NaN) à l’aide de la méthode spécifiée. Dans ce cas, les données vides sont remplies par des moyens.

```
[6]: data2= data.fillna(data['age'].mean())
print(data2.describe())
data2.shape
```

	pclass	survived	age	fare
count	1309.000000	1309.000000	1309.000000	1309.000000
mean	2.294882	0.381971	29.881135	33.292871
std	0.837836	0.486055	12.883199	51.738965
min	1.000000	0.000000	0.166700	0.000000
25%	2.000000	0.000000	22.000000	7.895800
50%	3.000000	0.000000	29.881135	14.454200
75%	3.000000	1.000000	35.000000	31.275000
max	3.000000	1.000000	80.000000	512.329200

```
[6]: (1309, 5)
```

Ou nous les éliminerons tous.

```
[7]: data = data.dropna(axis=0)
print(data.describe())
data.shape
```

	pclass	survived	age	fare
count	1045.000000	1045.000000	1045.000000	1045.000000
mean	2.206699	0.408612	29.851834	36.686080
std	0.841542	0.491813	14.389201	55.732533
min	1.000000	0.000000	0.166700	0.000000
25%	1.000000	0.000000	21.000000	8.050000
50%	2.000000	0.000000	28.000000	15.750000
75%	3.000000	1.000000	39.000000	35.500000
max	3.000000	1.000000	80.000000	512.329200

```
[7]: (1045, 5)
```

2.3 Statistics avec Groupby() et value_counts()

Une opération groupby implique une combinaison de fractionnement de l'objet, d'application d'une fonction et de combinaison des résultats. Cela peut être utilisé pour regrouper de grandes quantités de données et d'opérations de calcul sur ces groupes.

```
[8]: data.groupby(['sex']).mean()
```

```
[8]:
```

	pclass	survived	age	fare
sex				
female	2.048969	0.752577	28.687071	50.308775
male	2.299848	0.205479	30.539701	28.641019

On observe que plus de femmes ont survécu au naufrage du Titanic.

```
[9]: data.groupby(['sex', 'pclass']).mean()
```

```
[9]:
```

		survived	age	fare
sex	pclass			
female	1	0.962406	37.037594	111.996900
	2	0.893204	27.499191	23.267395
	3	0.473684	22.185307	14.655758
male	1	0.350993	41.029250	74.818213
	2	0.145570	30.815401	20.934335
	3	0.169540	25.863027	12.103374

```
[10]: data['pclass'].value_counts()
```

```
[10]: 3    500
      1    284
      2    261
      Name: pclass, dtype: int64
```

```
[11]: data[data['age'] < 18]['pclass'].value_counts()
```

```
[11]: 3    106
      2     33
      1     15
      Name: pclass, dtype: int64
```

```
[12]: data[data['age']>18].groupby(['sex', 'pclass']).mean()
```

```
[12]:
```

		survived	age	fare
sex	pclass			
female	1	0.966667	39.358333	111.502258
	2	0.878049	32.067073	22.482470
	3	0.436170	29.457447	13.448982
male	1	0.328671	42.716783	71.890794
	2	0.087591	34.069343	19.254531
	3	0.159420	29.688406	10.042917

2.4 Operation sur les séries

Ndarray unidimensionnel avec étiquettes d'axe (y compris les séries chronologiques).

Les étiquettes n'ont pas besoin d'être uniques, mais doivent être de type hachable. L'objet prend en charge l'indexation basée à la fois sur les entiers et sur les étiquettes et fournit une multitude de méthodes pour effectuer des opérations impliquant l'index. Les méthodes statistiques de ndarray ont été remplacées pour exclure automatiquement les données manquantes (actuellement représentées par NaN).

```
[13]: #data['age'] est une serie
print(data['age'][0:10]) # slicing a serie
data2 = data['age']> 18 # cree un mask
print(data2)
print(data[data['age']< 18]) # boolean indexing
```

```
0    29.0000
1     0.9167
2     2.0000
3    30.0000
4    25.0000
5    48.0000
6    63.0000
7    39.0000
8    53.0000
9    71.0000
Name: age, dtype: float64
0      True
1     False
2     False
3      True
4      True
```

```

...
1301    True
1304   False
1306    True
1307    True
1308    True
Name: age, Length: 1045, dtype: bool
   pclass  survived    sex    age    fare
1         1         1  male  0.9167 151.5500
2         1         0 female  2.0000 151.5500
53        1         0  male 17.0000  47.1000
54        1         1  male 11.0000 120.0000
55        1         1 female 14.0000 120.0000
...      ...      ...    ...    ...
1265      3         0 female 10.0000  24.1500
1275      3         0  male 16.0000  18.0000
1279      3         0 female 14.0000   7.8542
1300      3         1 female 15.0000  14.4542
1304      3         0 female 14.5000  14.4542

```

[154 rows x 5 columns]

```
[14]: data.head()
```

```

[14]:   pclass  survived    sex    age    fare
0         1         1 female 29.0000 211.3375
1         1         1  male  0.9167 151.5500
2         1         0 female  2.0000 151.5500
3         1         0  male 30.0000 151.5500
4         1         0 female 25.0000 151.5500

```

2.5 Méthodes loc et iloc

Les méthodes loc et iloc sont des méthodes Pandas essentielles utilisées pour filtrer, sélectionner et manipuler des données. Ils nous permettent d'accéder à la combinaison souhaitée de lignes et de colonnes.

La principale différence entre eux est la façon dont ils accèdent aux lignes et aux colonnes :

- loc utilise des étiquettes de ligne et de colonne.
- iloc utilise des index de ligne et de colonne.

```

[15]: print(data.iloc[3,3]) # iloc pour la localisation d'index récupère la valeur de
      ↪ la 4ème ligne et de la 4ème colonne
print(data.iloc[0,3]) # récupère la valeur de la première ligne et de la 4ème
      ↪ colonne
print(data.loc[0:2, 'age'])

```

30.0

```
29.0
0    29.0000
1     0.9167
2     2.0000
Name: age, dtype: float64
```

2.6 Codefication des donnees

Dans cette partie nous allons créer des catégories et codifier les donnees string avec les fonctions `map()`, `replace()` et `cat.codes`.

1. Fonction `map` :

La fonction `map` en Pandas est utilisée pour appliquer une fonction ou un dictionnaire à chaque élément d’une colonne d’un DataFrame. Elle est souvent utilisée pour transformer les valeurs existantes en de nouvelles valeurs. Par exemple, si vous avez une colonne “Genre” avec des valeurs “M” et “F”, vous pouvez utiliser `map` pour la convertir en “Masculin” et “Féminin” en spécifiant un dictionnaire de correspondance. Voici un exemple :

```
# Créer un dictionnaire de correspondance
correspondance = {'M': 'Masculin', 'F': 'Féminin'}

# Appliquer la fonction map à la colonne 'Genre'
df['Genre'] = df['Genre'].map(correspondance)
```

2. Fonction `replace` :

La fonction `replace` est utilisée pour remplacer les valeurs spécifiées dans une colonne par de nouvelles valeurs. Vous pouvez spécifier les valeurs à remplacer et les nouvelles valeurs à l’aide d’un dictionnaire. Par exemple, pour remplacer toutes les occurrences de “Oui” par 1 et “Non” par 0 dans une colonne “Achat”, vous pouvez utiliser `replace` de la manière suivante :

```
# Remplacer les valeurs dans la colonne 'Achat'
df['Achat'] = df['Achat'].replace({'Oui': 1, 'Non': 0})
```

3. Fonction `cat` :

La fonction `cat` est principalement utilisée pour concaténer (joindre) des DataFrames le long d’un axe particulier, que ce soit en ligne ou en colonne. Elle est utile lorsque vous avez plusieurs DataFrames que vous souhaitez fusionner en un seul. Par exemple, pour concaténer deux DataFrames `df1` et `df2` en ajoutant les lignes du second à la fin du premier, vous pouvez utiliser `concat` de la manière suivante :

```
# Concaténer les DataFrames df1 et df2 en ajoutant les lignes de df2 à df1
resultat = pd.concat([df1, df2])
```

Ces fonctions sont couramment utilisées pour la manipulation de données dans Pandas et peuvent être très utiles pour préparer et nettoyer vos données.

Pour créer des catégories d’âge comme indiqué dans l’exemple, il existe deux méthodes, nous utilisons soit l’indexation booléenne, soit la fonction de carte qui est illustrée ci-dessous

```
[16]: df= data.copy()
df.loc[data['age']< 20,'age']=0
df.loc[(data['age']>= 20) & ( data['age']< 30),'age']=1
df.loc[(data['age']>= 30) & (data['age']< 40),'age']=2
df.loc[data['age']>= 40,'age']=3
df.head()
```

```
[16]:
```

	pclass	survived	sex	age	fare
0	1	1	female	1.0	211.3375
1	1	1	male	0.0	151.5500
2	1	0	female	0.0	151.5500
3	1	0	male	2.0	151.5500
4	1	0	female	1.0	151.5500

```
[17]: df['age'].value_counts()
```

```
[17]: 1.0    344
3.0    244
2.0    232
0.0    225
Name: age, dtype: int64
```

```
[18]: df.groupby(['age']).mean()
```

```
[18]:
```

	pclass	survived	fare
age			
0.0	2.542222	0.471111	30.837371
1.0	2.436047	0.369186	26.739255
2.0	2.103448	0.422414	41.286781
3.0	1.672131	0.393443	51.728314

La fonction map() applique une fonction sur tous les elements d'une colonne.

```
[19]: df['age'].map(lambda x:x+2)
```

```
[19]: 0      3.0
1      2.0
2      2.0
3      4.0
4      3.0
...
1301    5.0
1304    2.0
1306    3.0
1307    3.0
1308    3.0
Name: age, Length: 1045, dtype: float64
```



```
[20]: def category_ages(age):
      if age <= 20:
          return '<20 ans'
      elif (age > 20) & (age <= 30):
          return '20-30 ans'
      elif (age > 30) & (age <= 40):
          return '30-40 ans'
      else:
          return '+40 ans'
```

```
[21]: df['age'] = df['age'].map(category_ages)
```

```
[22]: df.head()
```

```
[22]:   pclass  survived    sex    age    fare
0        1         1  female <20 ans  211.3375
1        1         1   male <20 ans  151.5500
2        1         0  female <20 ans  151.5500
3        1         0   male <20 ans  151.5500
4        1         0  female <20 ans  151.5500
```

De plus, ici, la colonne sex contient des informations au format String, ce qui est inhabituel et inadapté à une application d'apprentissage automatique. Ainsi, nous utiliserons une codification pour convertir cette colonne au format numérique. Pour cette tâche, il existe trois méthodes adaptées à cette tâche : la fonction map(), replace() et cat.codes

```
[23]: def Conv_sex_to_number(sex):
      if sex == 'female':
          return 0
      else :
          return 1
df['sex'] = df['sex'].map(Conv_sex_to_number)
df.head()
```

```
[23]:   pclass  survived  sex    age    fare
0        1         1   0 <20 ans  211.3375
1        1         1   1 <20 ans  151.5500
2        1         0   0 <20 ans  151.5500
3        1         0   1 <20 ans  151.5500
4        1         0   0 <20 ans  151.5500
```

```
[24]: df['sex'] = data['sex'].map({'male':0, 'female':1})
df.head()
```

```
[24]:   pclass  survived  sex    age    fare
0        1         1   1 <20 ans  211.3375
1        1         1   0 <20 ans  151.5500
```

2	1	0	1	<20 ans	151.5500
3	1	0	0	<20 ans	151.5500
4	1	0	1	<20 ans	151.5500

```
[25]: df['sex'] = data['sex'].replace(['male','female'],[0,1])
df.head()
```

```
[25]:
```

	pclass	survived	sex	age	fare
0	1	1	1	<20 ans	211.3375
1	1	1	0	<20 ans	151.5500
2	1	0	1	<20 ans	151.5500
3	1	0	0	<20 ans	151.5500
4	1	0	1	<20 ans	151.5500

```
[26]: df['sex']= data['sex'].astype('category').cat.codes
df.head()
```

```
[26]:
```

	pclass	survived	sex	age	fare
0	1	1	0	<20 ans	211.3375
1	1	1	1	<20 ans	151.5500
2	1	0	0	<20 ans	151.5500
3	1	0	1	<20 ans	151.5500
4	1	0	0	<20 ans	151.5500

3 Tracés les plus utiles

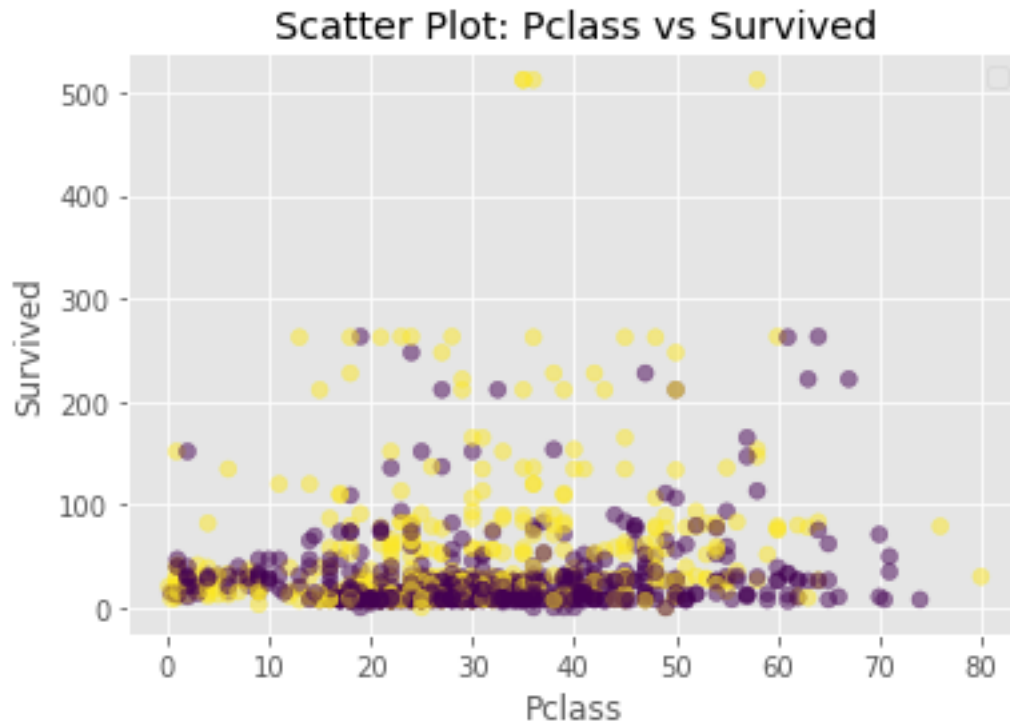
3.1 Scatter Plot: Nuage de points

Le nuage de points est idéal pour explorer les relations et les corrélations entre deux variables. Il affiche chaque paire de points de données dans un espace bidimensionnel, permettant d'observer les tendances, les groupements ou les dispersions qui peuvent indiquer des liens entre les variables.

```
[27]: plt.scatter(data['age'], data['fare'],c=data['survived'], alpha=0.5)
plt.xlabel('Pclass')
plt.ylabel('Survived')
plt.legend()
plt.title('Scatter Plot: Pclass vs Survived')
```

No handles with labels found to put in legend.

```
[27]: Text(0.5, 1.0, 'Scatter Plot: Pclass vs Survived')
```

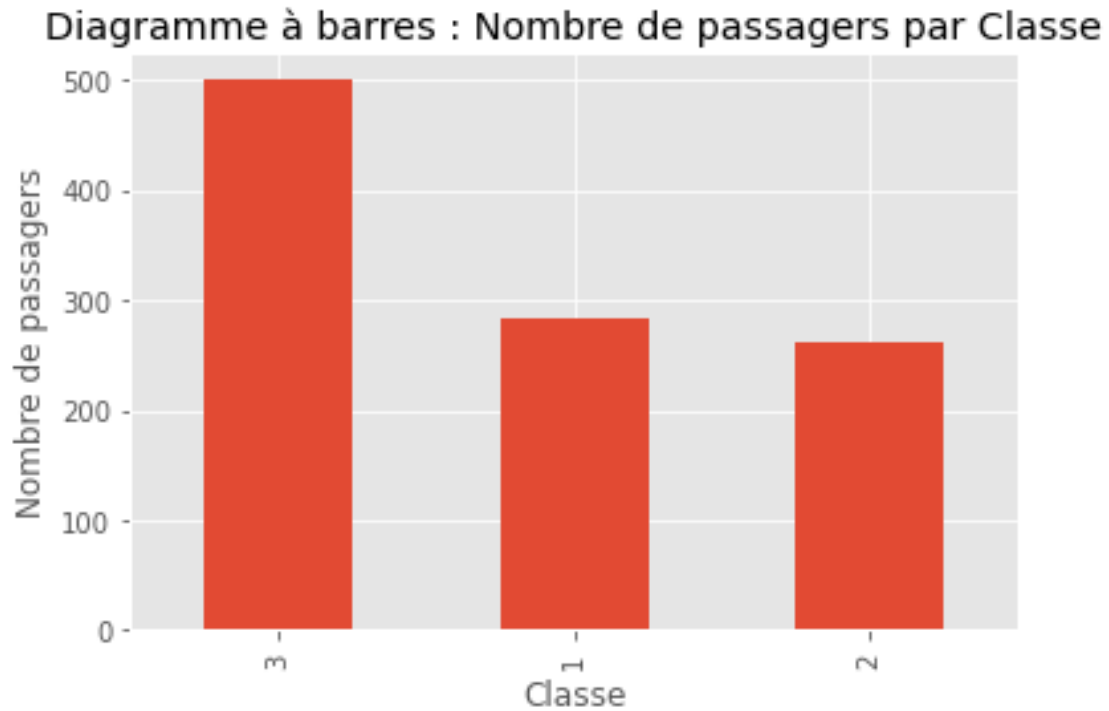


3.2 Bar Plot: Diagramme à barres

Le diagramme à barres est efficace pour comparer des données catégorielles entre différents groupes ou catégories. Il affiche des barres verticales ou horizontales pour représenter la fréquence, la proportion ou la valeur agrégée de chaque catégorie, facilitant les comparaisons visuelles.

```
[28]: data['pclass'].value_counts().plot(kind='bar')
plt.xlabel('Classe')
plt.ylabel('Nombre de passagers')
plt.title('Diagramme à barres : Nombre de passagers par Classe')
```

```
[28]: Text(0.5, 1.0, 'Diagramme à barres : Nombre de passagers par Classe')
```

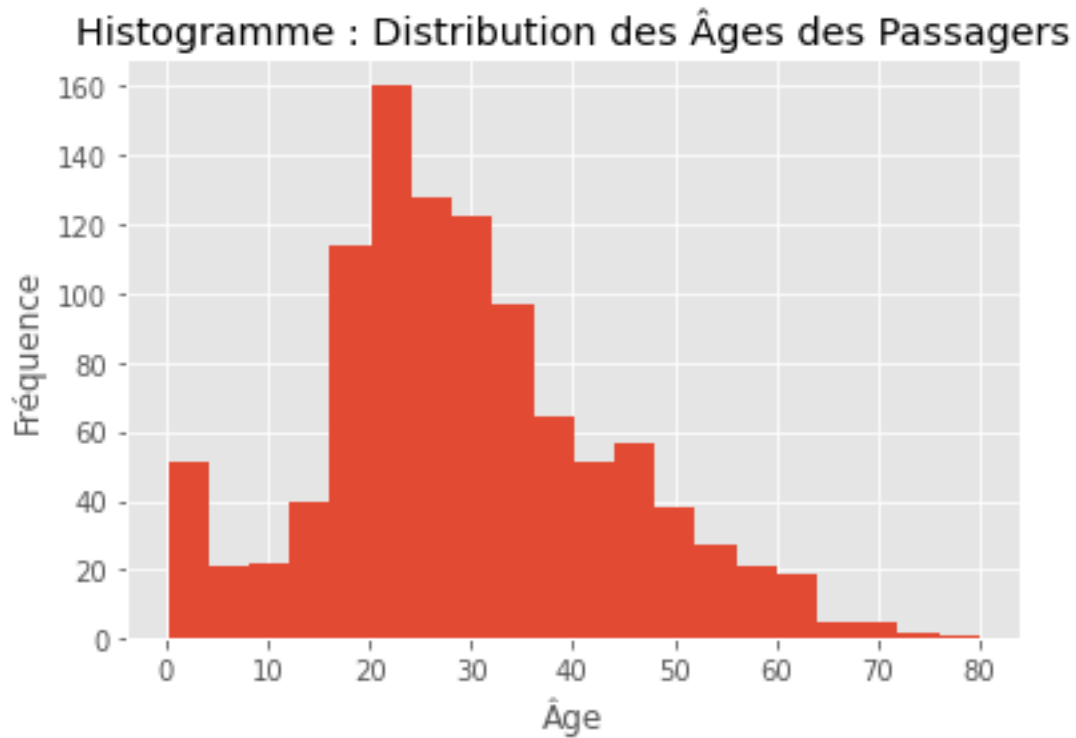


3.3 Histogram: Histogramme

L'histogramme est idéal pour visualiser la distribution d'une variable unique. Il divise la plage de valeurs en intervalles et affiche la fréquence ou la densité des observations dans chaque intervalle, permettant d'identifier les modes, les dispersions et les formes de distribution.

```
[29]: data['age'].hist(bins=20)
      plt.xlabel('Âge')
      plt.ylabel('Fréquence')
      plt.title('Histogramme : Distribution des Âges des Passagers')
```

```
[29]: Text(0.5, 1.0, 'Histogramme : Distribution des Âges des Passagers')
```

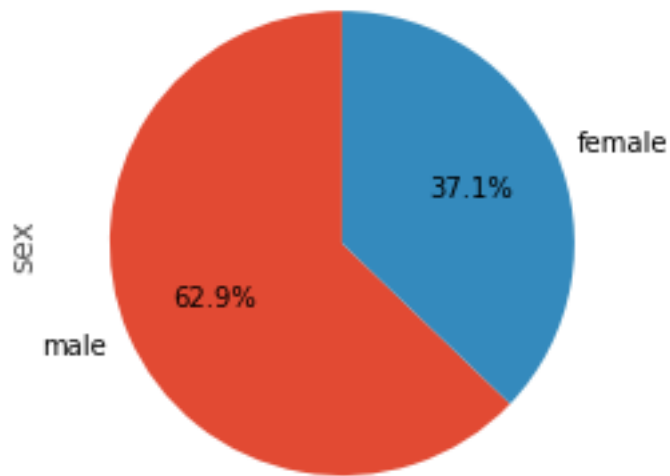


3.4 Pie Chart: Diagramme circulaire

Le diagramme circulaire est utilisé pour montrer la composition d'un ensemble en termes de pourcentages. Il divise un cercle en secteurs proportionnels aux fréquences relatives des catégories, fournissant une vue globale des parts de chaque catégorie dans le tout.

```
[30]: data['sex'].value_counts().plot(kind='pie', autopct='%1.1f%%', startangle=90)
```

```
[30]: <AxesSubplot:ylabel='sex'>
```

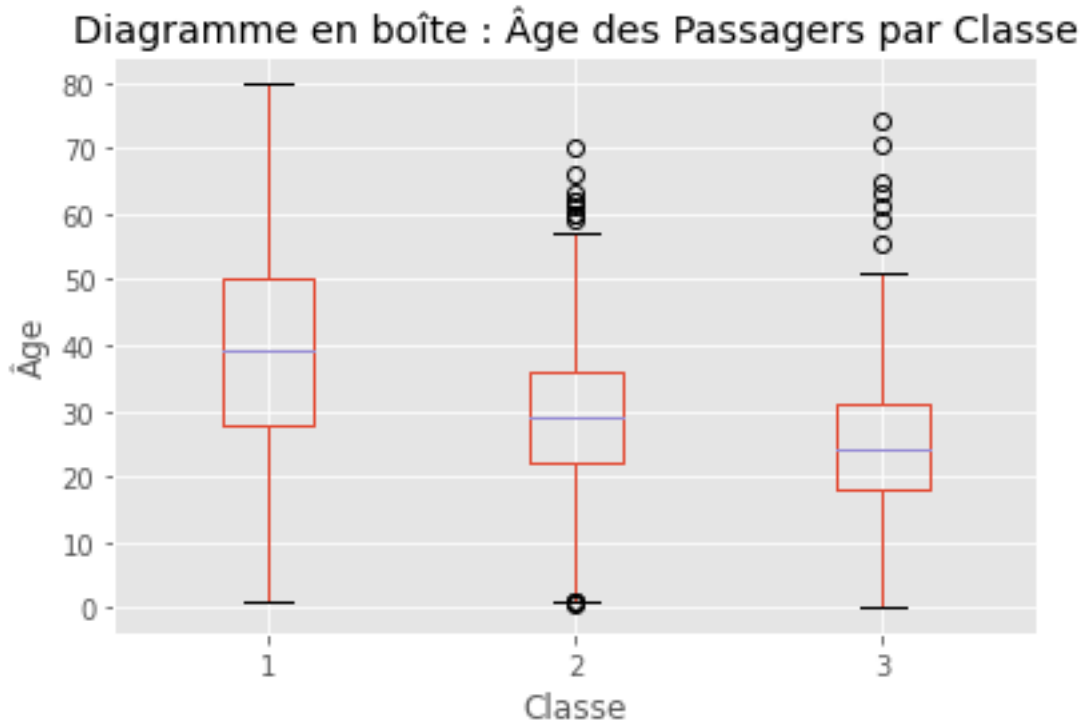


3.5 Box Plot:

Diagramme en boîte Le diagramme en boîte est utile pour visualiser la distribution des données et identifier les valeurs aberrantes. Il affiche les quartiles, la médiane et les points de données potentiellement aberrants, aidant à détecter les tendances centrales, les dispersions et les formes de distribution.

```
[31]: data.boxplot(column='age', by='pclass')
plt.xlabel('Classe')
plt.ylabel('Âge')
plt.title('Diagramme en boîte : Âge des Passagers par Classe')
plt.suptitle('') # Supprimer le titre automatique ajouté par pandas
```

```
[31]: Text(0.5, 0.98, '')
```



4 Tracés les plus utiles de Seaborn

Seaborn est une bibliothèque Python de visualisation de données basée sur matplotlib. Elle fournit une interface de haut niveau pour dessiner des graphiques statistiques attrayants et informatifs.

```
[32]: import seaborn as sns
```

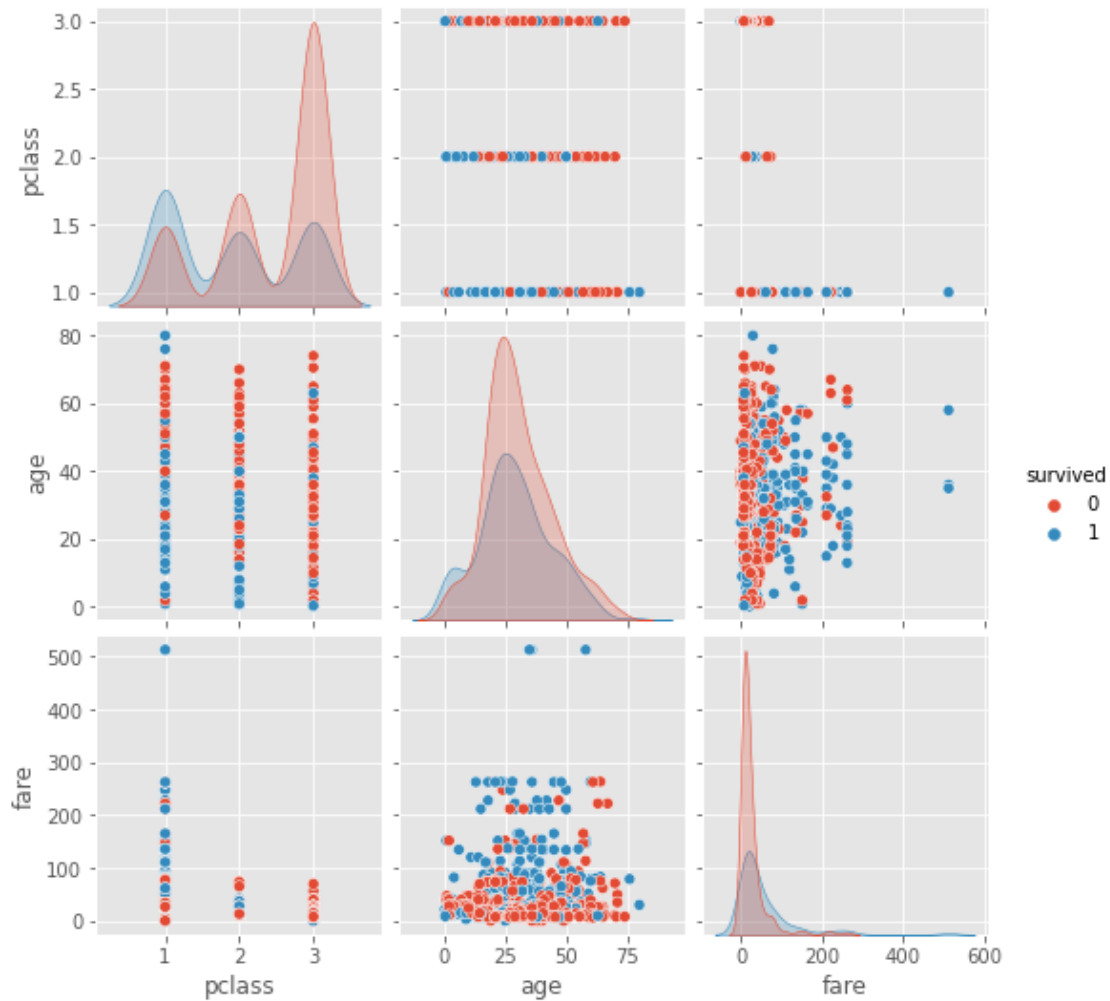
4.1 La vue d'ensemble Pairplot()

Tracer des relations par paires dans un jeu de données. Par défaut, cette fonction créera une grille d'axes telle que chaque variable numérique dans les données sera partagée sur les axes y sur une seule ligne et les axes x sur une seule colonne. Les diagrammes diagonaux sont traités différemment : un diagramme de distribution univariée est tracé pour montrer la distribution marginale des données dans chaque colonne.

Il est également possible d'afficher un sous-ensemble de variables ou de tracer différentes variables sur les lignes et les colonnes.

```
[33]: sns.pairplot(data, hue='survived')
```

```
[33]: <seaborn.axisgrid.PairGrid at 0x7fb6d0dcd4c0>
```

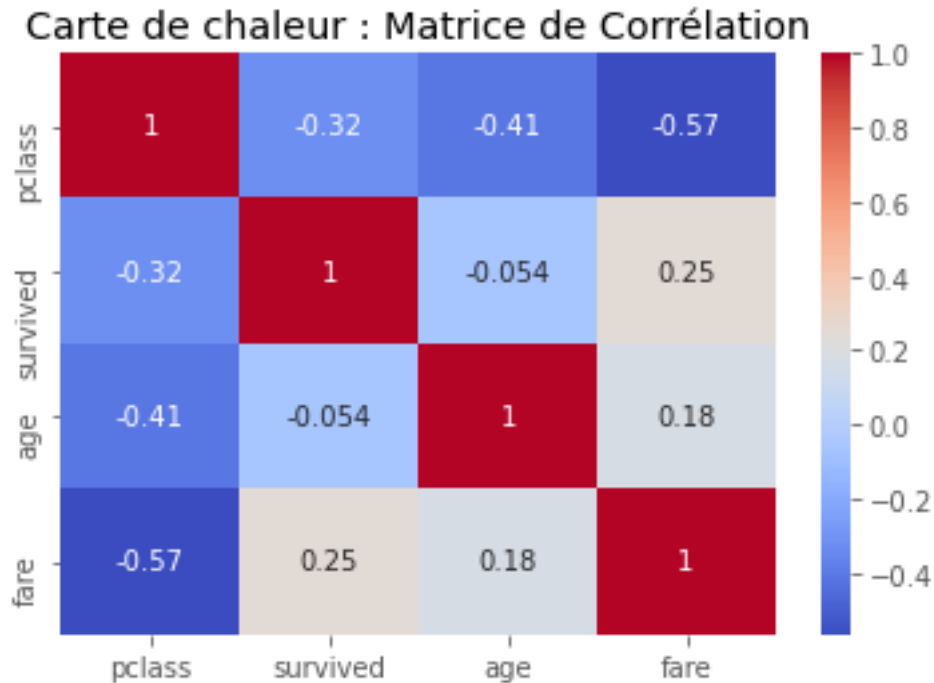


4.2 Heatmap: Carte de chaleur

Description: La carte de chaleur est utile pour afficher une matrice de valeurs avec une intensité codée en couleurs. Elle est souvent utilisée pour représenter les corrélations entre les variables ou pour visualiser les données tabulaires, montrant les tendances et les schémas qui se dégagent.

```
[34]: sns.heatmap(data.corr(), annot=True, cmap='coolwarm')
plt.title('Carte de chaleur : Matrice de Corrélations')
```

```
[34]: Text(0.5, 1.0, 'Carte de chaleur : Matrice de Corrélations')
```

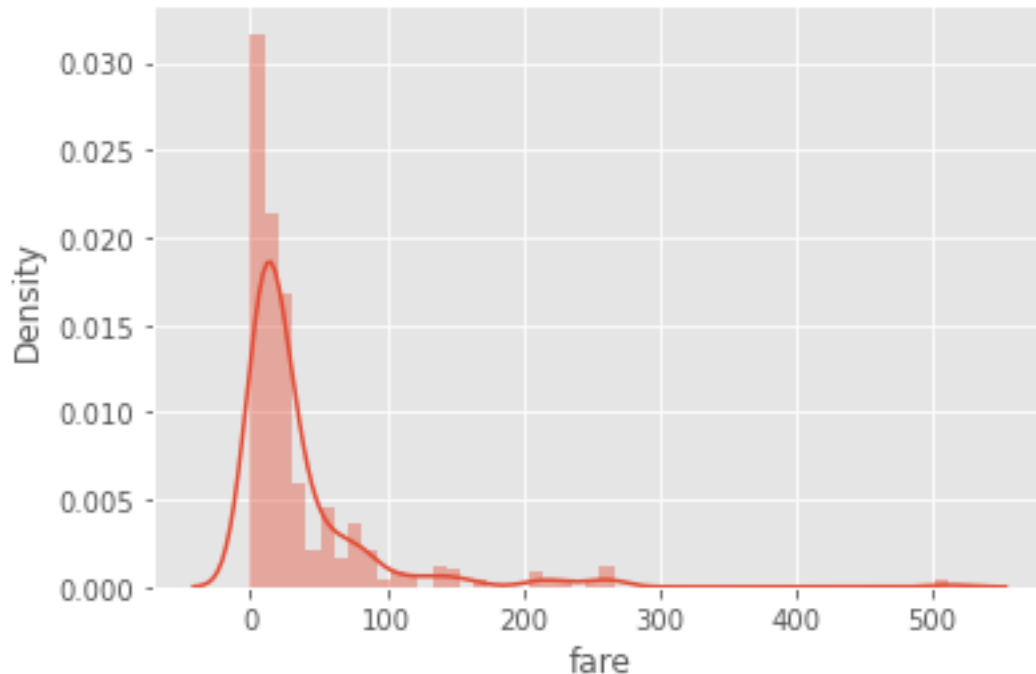
4.3 Distplot (Distribution Plot) : Tracé de Distribution

Description : Le tracé de distribution est une visualisation puissante pour représenter la distribution univariée d'une variable. Il combine un histogramme avec une estimation de la densité des noyaux (KDE) pour afficher la fréquence relative des valeurs dans un intervalle et la forme de la distribution sous-jacente. Cela permet d'identifier les modes, les asymétries et les tendances générales de la variable.

```
[35]: sns.distplot(data['fare'])
```

```
/Users/mac/opt/anaconda3/lib/python3.8/site-
packages/seaborn/distributions.py:2551: FutureWarning: `distplot` is a
deprecated function and will be removed in a future version. Please adapt your
code to use either `displot` (a figure-level function with similar flexibility)
or `histplot` (an axes-level function for histograms).
  warnings.warn(msg, FutureWarning)
```

```
[35]: <AxesSubplot:xlabel='fare', ylabel='Density'>
```



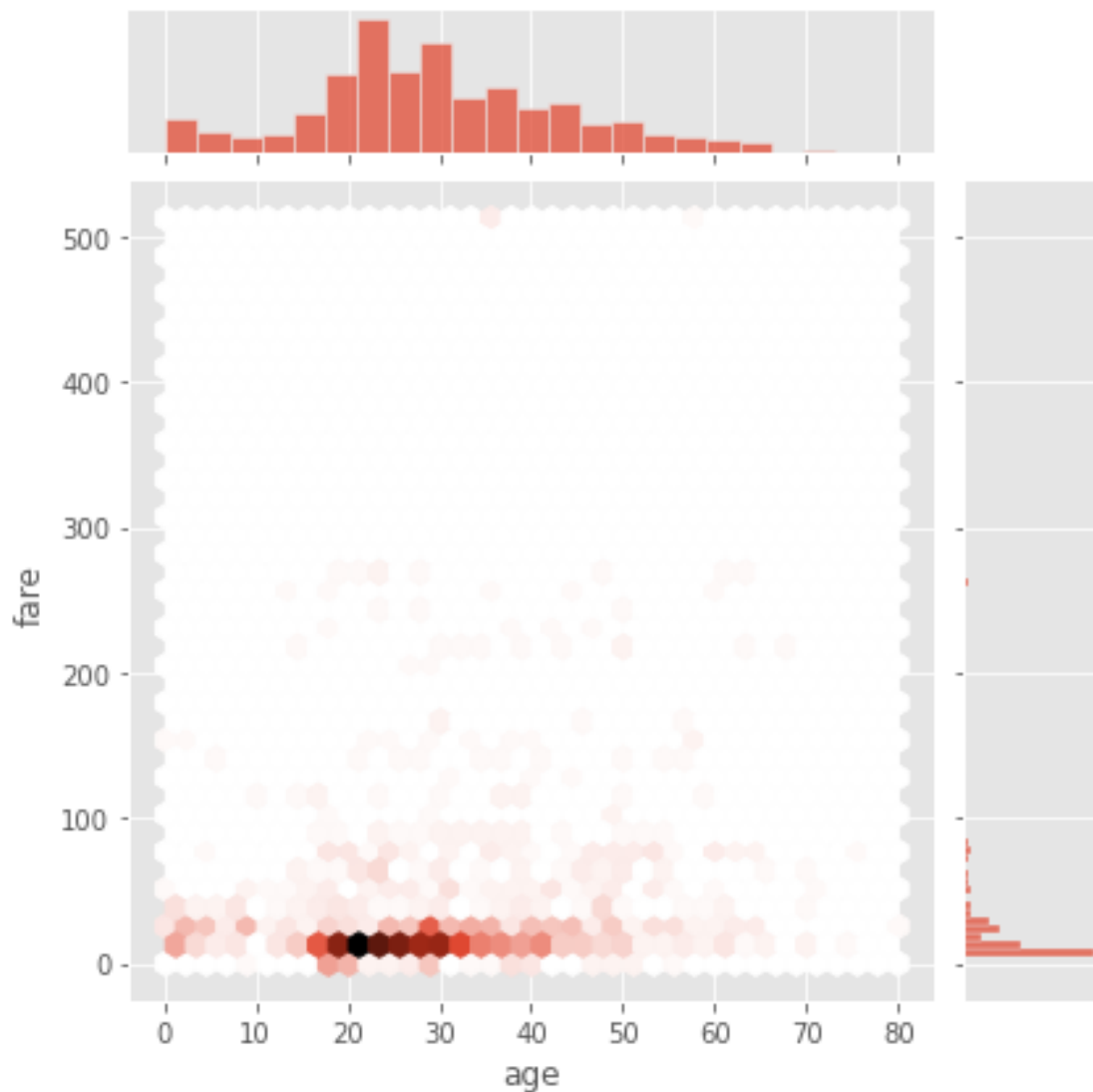
4.4 Jointplot : Tracé Conjoints

Description : Le tracé conjoint est utilisé pour explorer la relation entre deux variables en affichant simultanément un nuage de points et des tracés de distribution marginaux pour chaque variable. Cela permet d'analyser la corrélation, la dispersion et la distribution des deux variables en une seule visualisation. Il offre une vue complète des interactions entre les variables, mettant en évidence les tendances et les concentrations.

```
[36]: sns.jointplot('age', 'fare', data=data, kind='hex')
```

```
/Users/mac/opt/anaconda3/lib/python3.8/site-packages/seaborn/_decorators.py:36:
FutureWarning: Pass the following variables as keyword args: x, y. From version
0.12, the only valid positional argument will be `data`, and passing other
arguments without an explicit keyword will result in an error or
misinterpretation.
  warnings.warn(
```

```
[36]: <seaborn.axisgrid.JointGrid at 0x7fb6d5dbf730>
```



5 Exercice :

Une entreprise automobile chinoise, Geely Auto, nourrit l'ambition d'entrer sur le marché américain en établissant une unité de production locale afin de rivaliser avec les constructeurs automobiles américains et européens. Dans cette optique, elle a fait appel à une société de conseil spécialisée dans le secteur automobile pour appréhender les facteurs qui influencent le prix des voitures sur ce marché spécifique. Cette démarche vise à cerner avec précision les éléments qui impactent les tarifs des véhicules, en prenant en compte les spécificités potentiellement différentes du marché américain par rapport à celui de la Chine. La société de conseil s'est fixé deux objectifs clés :

1. Identifier les variables significatives pour prédire le prix des voitures.
2. Évaluer dans quelle mesure ces variables parviennent à expliquer le prix des voitures.

Travail à réaliser :

Dans cette étude, nous sommes chargés de mener une analyse exploratoire des données relatives aux prix des voitures en relation avec les variables indépendantes disponibles. Cette analyse sera utilisée par la direction de l'entreprise pour mieux appréhender la manière dont les prix varient en fonction des variables indépendantes. De ce fait, elle pourra ajuster la conception des véhicules ainsi que la stratégie commerciale, entre autres, pour atteindre des niveaux de prix spécifiques. En outre, le modèle résultant de cette analyse constituera un outil précieux pour la direction, lui permettant de saisir les dynamiques sous-jacentes aux variations des prix sur un nouveau marché. Les noms des colonnes de `df_auto.columns` représentent différentes caractéristiques des données relatives aux voitures. Voici une explication en français pour chacune des colonnes :

- `car_ID` : Identifiant unique associé à chaque voiture dans le jeu de données.
- `symboling` : Niveau de notation de risque attribué à la voiture par les compagnies d'assurance.
- `CarName` : Le nom de la marque et du modèle de la voiture.
- `fueltype` : Type de carburant utilisé par la voiture (essence ou diesel).
- `aspiration` : Type d'aspiration du moteur (atmosphérique ou turbo).
- `doornumber` : Nombre de portes de la voiture (deux ou quatre portes).
- `carbody` : Type de carrosserie de la voiture (berline, coupé, hayon, etc.).
- `drivewheel` : Type de transmission des roues motrices (avant, arrière ou quatre roues motrices).
- `engineloation` : Emplacement du moteur dans la voiture (avant ou arrière).
- `wheelbase` : Empattement de la voiture (distance entre les essieux avant et arrière).
- `carlength` : Longueur de la voiture.
- `carwidth` : Largeur de la voiture.
- `carheight` : Hauteur de la voiture.
- `curbweight` : Poids à vide de la voiture.
- `enginetype` : Type de moteur (en ligne, en V, etc.).
- `cylindernumber` : Nombre de cylindres du moteur.
- `enginesize` : Taille du moteur en termes de volume de cylindrée.
- `fuelsystem` : Système de carburant du moteur (injection, carburateur, etc.).
- `boreratio` : Rapport d'alésage du moteur.
- `stroke` : Course du moteur.
- `compressionratio` : Taux de compression du moteur.
- `horsepower` : Puissance du moteur en chevaux.
- `peakrpm` : Régime moteur auquel la puissance maximale est obtenue.
- `citympg` : Consommation de carburant en miles par gallon en milieu urbain.
- `highwaympg` : Consommation de carburant en miles par gallon sur autoroute.
- `price` : Prix de la voiture.

Ces colonnes contiennent des informations variées sur les voitures, allant des détails techniques aux attributs liés au design et au prix.

À partir de cette analyse exploratoire des données, nous devons répondre aux questions suivantes :

1. Identification des attributs :

- Variable cible (target) :
- Nombre de lignes et de colonnes :
- Types de variables qualitatives et quantitatives :
- Présence de valeurs manquantes :

2. Visualisation avec un diagramme en boîte (box plot) et un tracé de distribution (distplot) de la variable cible :

3. Traitement de la colonne “CarName” :

Nous mettrons en œuvre le code suivant afin de conserver uniquement le nom de marque des voitures dans la colonne “CarName” :

```
df['CarName'] = df['CarName'].str.split(' ', expand=True).loc[:, 0]
```

Ensuite, nous devons examiner s’il y a des erreurs de frappe dans les noms de marques et les corriger si nécessaire, en utilisant la fonction `replace`.

4. Tracé d’un histogramme du nombre de voitures par marque :

5. Tracé de boîtes à moustaches pour les variables : doornumber, fuelsystem, fueltype, aspiration, carbody, enginelocation, drivewheel, enginetype, cylindernumber :

- Quel type est le plus fréquent ?
- Quel type est le plus coûteux ?

6. Visualisation des relations entre ces variables et le prix :

- Les véhicules diesel ou essence sont-ils les plus onéreux ?
- Quel type de carrosserie présente le coût le plus élevé ?

7. Visualisation du prix en fonction des variables qualitatives et réponses aux questions :

- Quelle variable semble présenter une corrélation positive avec le prix ?
- Quelle variable ne semble pas afficher de corrélation avec le prix ?
- Quelle variable semble être en corrélation négative avec le prix ?

8. Selon votre expertise, quelles variables sont les plus déterminantes pour prédire le prix ?